

Correction de la serie N01 : SECURITE & CRYPTOGRAPHIE

EXERCICE01 :

Programme en java qui permet se produire une permutations aleatoire pour un alphabet donne en ligne de commande :

```
import java.sql.Array;
import java.util.*;

public class Main {
    public static String permute(String Alphbet){
        char []ArrayAlphbet=new char[Alphbet.length()];
        for (int i=0;i<Alphbet.length();i++){
            ArrayAlphbet[i]=Alphbet.charAt(i);
        }

        Random rnd = new Random();
        for (int i = ArrayAlphbet.length - 1; i > 0; i--) {
            int index = rnd.nextInt(i + 1);
            char temp = ArrayAlphbet[index];
            ArrayAlphbet[index] = ArrayAlphbet[i];
            ArrayAlphbet[i] = temp;
        }
        String newString = String.valueOf(ArrayAlphbet);
        return newString;
    }
    public static void main(String[] args) {
        if(args.length !=1){
            System.out.println("please enter a string alphabet");
            System.exit(0);
        }
        //      String Alphbet="ABC";
        String Alphbet=args[0];
        String newString=permute(Alphbet);
        System.out.println(newString);
    }
}
```

EXERCICE02:

Programme en java qui produit toutes les permutations possibles pour un alphabet donne en ligne de commande :

```
public class Main {
    public static void main(String[] args) {
```

```

if (args.length != 2 || !args[0].equals("-a")) {
    System.out.println("please enter a string alphabet");
    System.exit(0);
}
// String Alphbet = "abc";
String Alphbet = args[1];
char[] ArrayAlphbet = new char[Alphbet.length()];
for (int i = 0; i < Alphbet.length(); i++) {
    ArrayAlphbet[i] = Alphbet.charAt(i);
}
char temp;
String newString;
String orginString = String.valueOf(ArrayAlphbet);
int i = 1;

while (i < ArrayAlphbet.length - 1) {
    temp = ArrayAlphbet[i];
    ArrayAlphbet[i] = ArrayAlphbet[i + 1];
    ArrayAlphbet[i + 1] = temp;
    newString = String.valueOf(ArrayAlphbet);
    System.out.println(newString);
    i++;
    if (i == ArrayAlphbet.length - 1) {
        while (i > 1) {
            i--;
            temp = ArrayAlphbet[i];
            ArrayAlphbet[i] = ArrayAlphbet[i - 1];
            ArrayAlphbet[i - 1] = temp;
            newString = String.valueOf(ArrayAlphbet);
            System.out.println(newString);

            if (orginString.equals(newString)) {
                i = ArrayAlphbet.length;
                break;
            }
        }
    }
}
}
}

```

EXERCICE03:

programme en C qui permet de générer d'une façon exhaustive tous les mots de passe composés par les caractères d'un alphabet donné. Le programme a des options à l'exécution :

- a suivi d'une chaine de caractères pour introduire l'alphabe

-t suivi d'un nombre permet de fixer la taille des mots de passe à générer.

- m suivi d'un nombre permet de générer tous les mots de passe de taille inférieure ou égale au nombre saisi.

```
public class Main {
    public void ft(String A, String P, int t) {
        if (P.length() < t) {
            for (int i = 0; i < A.length(); i++) {
                String P2 = P + A.charAt(i);
                ft(A, P2, t);
            }
        } else {
            System.out.println(P);
        }
    }

    public void fm(String A, String P, int t) {
        if (P.length() < t) {
            for (int i = 0; i < A.length(); i++) {
                String P2 = P + A.charAt(i);
                System.out.println(P2);
                fm(A, P2, t);
            }
        }
    }

    public void generateAll(String A, int maxLen) {
        for (int i = 1; i <= maxLen; i++) {
            ft(A, "", i);
        }
    }

    public static void main(String[] args) {
        String alphabet = "";
        int len = 0;
        int maxLen = 0;
        boolean modeMaxLen = false;

        for (int i = 0; i < args.length; i++) {
            switch (args[i]) {
                case "-a":
                    if (i + 1 < args.length) {
                        alphabet = args[i + 1];
                        i++;
                    } else {
                        System.err.println(" -a suivi d'une chaine de caracteres");
                        return;
                    }
                    break;
                case "-t":
```

```

        if (i + 1 < args.length) {
            try {
                len = Integer.parseInt(args[i + 1]);
                i++;
            } catch (NumberFormatException e) {
                System.err.println("-t suivi d'un nombre");
                return;
            }
        } else {
            System.err.println("-t suivi d'un nombre.");
            return;
        }
        break;
    case "-m":
        if (i + 1 < args.length) {
            try {
                maxLen = Integer.parseInt(args[i + 1]);
                modeMaxLen = true;
                i++;
            } catch (NumberFormatException e) {
                System.err.println("-m suivi d'un nombre.");
                return;
            }
        } else {
            System.err.println("-m suivi d'un nombre.");
            return;
        }
        break;
    default:
        System.err.println(" option not exist " + args[i]);
        return;
    }
}

if (alphabet.isEmpty()) {
    System.err.println("L'alphabet fourni avec l'option -a.");
    return;
}
if (len == 0 && !modeMaxLen) {
    System.err.println("La taille des mots de passe fournie avec
l'option -t ou -m.");
    return;
}

Main generator = new Main();

if (modeMaxLen) {
    generator.generateAll(alphabet, maxLen);
} else {
    generator.ft(alphabet, "", len);
}
}
}

```

EXERCICE04 :

programme en Java implémentant l'algorithme de César. Le programme doit prendre trois paramètres en entrée. Le premier paramètre indique l'opération à effectuer (-C pour chiffrer ou -D pour déchiffrer), le deuxième paramètre spécifie le fichier d'entrée, tandis que le troisième paramètre spécifie le fichier de sortie contenant le résultat après avoir appliqué la fonction demandée sur le fichier d'entrée.

```
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class Main {
    public static String encrypt(String text, int p){
        String result = "";
        for(int i=0;i<text.length();i++){
            char c=text.charAt(i);
            if(Character.isUpperCase(c)){
                result += (char) ((c+p-65)%26+65);
            }else if(Character.isLowerCase(c)){
                result += (char) ((c+p-97)%26+97);
            }else if(c == ' '){
                result += (char) ((c+p-7)%26+97);
            }else{
                result += c;
            }
        }
        return result;
    }

    public static String decrypt(String text, int p){
        String result = "";
        for(int i=0;i<text.length();i++){
            char c=text.charAt(i);
            if(Character.isUpperCase(c)){
                result += (char) ((c-p-65)%26+65);
            }else if(Character.isLowerCase(c)){
                result += (char) ((c-p-97)%26+97);
            }else if(c == ' '){
                result += (char) ((c-p-7)%26+97);
            }else{
                result += c;
            }
        }
        return result;
    }
}
```

```

public static void main(String[] args) {

    if (args.length != 3) {
        System.out.println("please enter text ");
        System.exit(0);
    }

    String operation = args[0];
    String inputFile = args[1];
    String outputFile = args[2];

    // String operation = "-C";
    // String inputFile = "src/fileInput";
    // String outputFile = "src/fileOutput";

    try {
        FileReader reader = new FileReader(inputFile);
        FileWriter writer = new FileWriter(outputFile);
        int p = 3;

        int c;
        while ((c = reader.read()) != -1) {
            String text = Character.toString((char) c);
            String result;

            if ("-C".equals(operation)) {
                result = encrypt(text, p);
            } else if ("-D".equals(operation)) {
                result = decrypt(text, p);
            } else {
                System.out.println("please enter -C or -D");
                return;
            }

            writer.write(result);
        }

        reader.close();
        writer.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}

```

EXERCICE05 :

Les fréquences des caractères en français																										
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
-	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
0.0	6.7	0.6	2.4	2.9	13.9	0.9	0.8	0.8	6.1	0.3	0	4.7	2.1	5.6	4.1	2.5	1.3	5.3	5.3	6.3	5.2	1.3	0	0.4	0.3	0.1

Les fréquences dans le Cryptogramme																										
-	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
1	3	0	0	6	1	4	1	1	1	2	0	4	2	1	3	3	11	2	0	3	1	2	0	1	1	0
1.7	5.3	0	0	10.7	1.7	10.7	1.7	1.7	1.7	3.5	0	7.1	3.5	1.7	5.3	5.3	18.6	3.5	0	5.3	1.7	3.5	0	1.7	1.7	0
9	6			7	9	7	9	9	9	7		4	7	9	6	6	0	7		6	9	7		9	9	

DOQUFLTLQMVFRQPITQHADLQ_TFDYQNDQDOQGJAOXQMVFRQEFPQAD LJFP

La plus grande fréquence dans le cryptogramme correspond aux espaces dans le message original

DO_UFLTL_MVFR_PIT_HADL TFDY_ND_DO_GJAOX_MVFR_EFP_ADLJFP

La deuxième fréquence C'est pour F et D, mais un des deux pourra être remplacé par E Soit

DO_UELTL_MVER_PIT_HADL TEDY_ND_DO_GJAOX_MVER_EEP_ADLJEP

Ou

EO_UFLTL_MVFR_PIT_HAEL TFEY_NE_EO_GJAOX_MVFR_EFP_AELJFP

Nous remarquons que l'extrait de la phrase NE_EO ne pourrait être juste, donc nous allons écarter le remplacement de D par E

Maintenant, D pourra être codé par A

DO_UELTL_MVER_PIT_HADL TEDY_ND_DO_GJAOX_MVER_EEP_ADLJEP

AO_UELTL_MVER_PIT_HAAL TEAY_NA_AO_GJAOX_MVER_EEP_AALJEP

Remplaçons L par T

AO_UETTT_MVER_PIT_HAAT TEAY_NA_AO_GJAOX_MVER_EEP_AATJEP

Remplaçons P par S

AO_UETTT_MVER_SIT_HAAT TEAY_NA_AO_GJAOX_MVER_EES_AATJES

Remplaçons T par I

AO_UETIT_MVER_SII_HAAT IEAY_NA_AO_GJAOX_MVER_EES_AATJES

Remplaçons O par N

AN_UEITIT_MVER_SII_HAAT IEAY_NA_AN_GJANX_MVER_EES_AATJES

Ici nous pouvons remarquer que « AN » n'est pas un mot du dictionnaire français donc soit c'est la lettre A qui n'est pas à sa place soit N,

Soit la lettre N devrait être un U, pour avoir le AU, là encore on trouvera un mot UA qui n'est pas vrai dans le dictionnaire français.

C'est donc la lettre A qui n'est pas à sa place. Par conséquent la lettre A doit être remplacée par U. Ce qui donne

UN_UEITIT_MVER_SII_HAUT IEUY_NU_UN_GJANX_MVER_EES_AUTJES

On peut également deviner que U est un P et que J est un R

UN_PETIT_MVER_SII_HAUT IEUY_NU_UN_GJANX_MVER_EES_AUTRES

De ce fait nous analysons l'extrait

EES_AUTRES

On devinera facilement que E est un L et que A est un A

UN_PETIT_MVER_SII_HAUT IEUY_NU_UN_GRANX_MVER_LES_AUTRES

Regardons maintenant le mot SII. La lettre I au milieu n'a pas été remplacée. Elle ne peut être que la lettre O ce qui donne le mot SOI.

UN_PETIT_MVER_SOI_HAUT IEUY_NU_UN_GRANX_MVER_LES_AUTRES

Analysons le mot NU :

Les mots avec deux caractères avec le dernier est un U sont

wu, vu, tu, su, ru, qu, ou, nu, mu, lu, eu, du, bu, au, Pu, Cu

Les caractères T, S, R, O, N, E, A, L sont déjà utilisés.

Reste wu, vu, qu, mu, du, bu, Pu, Cu,

L'usage de WU, MU, BU, PU, CU est peu fréquent dans la langue française.

=> Vu, QU, LU, DU, BU,

on peut même écarté la lettre D, parce que la suite c'est un UN, deux voyelles qui se suivent
DU UN

=> D'UN

reste alors Vu, QU, BU

Analysons maintenant le mot HAUT,

Les mots en français de 4 lettres qui finissent par AUT sont :

vaut, saut, haut, faut La lettre S a été déjà utilisée,

donc le H ici peut être

ou bien un

H => HAUT

ou bien un

V=> VAUT,

ou bien un

F => FAUT

Faut, ici est un verbe au 3ème personnel,

* Analysons maintenant le mot de 5 lettres _IEUY avec la première "_" et la dernière lettre "Y" ne sont pas encore dévoilées.

vieux, sieur, rieur, pieux, mieux, lieux, leur, lieue, cieus,

Les lettres S, R, E, L sont déjà utilisées.

Il reste donc les mots : vieux, pieux, mieux, cieus

Pour ces deux derniers mots, cherchons les combinaisons qui conviennent et portent un sens

Vaut	<u>vieux</u>	==> VAUT MIEUX
Faut	<u>pieux</u>	
Haut	<u>mieux</u>	
	<u>cieux</u>	

On déduit que H vaut V et _ vaut M et Y vaut X

UN_PETIT_MVER_SOI_VAUT_MIEUX_NU_UN_GRANX_MVER_LES_AUTRES

Maintenant combinant ce résultat avec les possibilités du codage du mot NU, nous aurons la combinaison qui porte un sens c'est VAUT MIEUX QU'UN par remplacement du caractère N par Q.

UN_PETIT_MVER_SOI_VAUT_MIEUX_QU_UN_GRANX_MVER_LES_AUTRES

Analysons le mot GRANX, ce mot est de 5 lettres avec la première " G " et la dernière lettre "X" ne sont pas dévoilées.

Urane, trans, prang, orant, grand, grana, frane, franc, crans

La lettres U, E, O, T, A, S, P sont déjà utilisées, reste donc deux mots grand,

Le caractère G code G et X code D. Ce qui donne

UN_PETIT_MVER_SOI_VAUT_MIEUX_QU_UN_GRAND_MVER_LES_AUTRES

Maintenant nous nous retrouvons avec un mot de 4 lettres qui est dupliqué deux fois dans la phrase avec seulement un seul caractère dévoilé. MVER

Il est donc facile de deviner que MVER est le mot CHEZ. Finalement, le message en clair est :

UN_PETIT_CHEZ_SOI_VAUT_MIEUX_QU_UN_GRAND_CHEZ_LES_AUTRES

Autrement, nous pourrions lister tous les mots de 4 lettres qui ont un E à la 3ème position :

DJED, COEF, CLEF, CIEL, CHEZ, CHER, CHEF, BREF, BOER, BLED, BIEF, BÉER, AXER, AXEL, AVEZ, AVEC, AREC, AMER, AMEL, ABER, DHEA

Un premier filtre consiste à éliminer les caractères déjà utilisés.

DJED, CIEL, CHEZ, CHER, CHEF

Le seul mot parmi ces 5 mots qui donne un sens à notre phrase est le mot CHEZ.

UN_PETIT_CHEZ_SOI_VAUT_MIEUX_QU_UN_GRAND_CHEZ_LES_AUTRES

Vous pouvez être aidés par le site web [https ://www.motsavec.com/](https://www.motsavec.com/)

Fin

youssef